实验二：OpenMP IVF-Flat 向量检索

本实验接着 CSR SpMV 作业，使用 SIFT1M 数据集和 Faiss 训练好的 IVF 索引。需要实现两种 OpenMP 检索方法，并编写 SIMD 距离函数。

向量检索中：一个查询可能要访问多个倒排列表，扫描大量候选，再从中选出最近的 k 个向量。查询较多时，可以把查询分给不同线程；只有一个查询时，则需要考虑怎样把它的计算拆开。这是本次实验要比较的两种并行方式。

1 检索过程

数据库中有 N 个 d 维向量。给定查询 q，按平方欧氏距离找出最近的 k 个向量：

$$D(q, x_i) = \sum_{j=0}^{d-1} (q_j - x_{ij})^2.$$

IVF 用 k-means 把数据库分成 L 个倒排列表，每个列表有一个中心。检索时，先找出距离查询最近的 `nprobe` 个中心，再扫描这些中心对应的列表，计算候选距离并选出 Top-k。

IVF-Flat 保存完整向量，候选距离可以直接计算。不过，真正的近邻可能落在没有访问的列表里，所以检索结果不一定与全库扫描相同。

设查询 q 选中的列表集合为 P(q)，其候选数为：

$$C(q) = \sum_{l \in P(q)} |I_l|.$$

距离计算量约为 `O((L+C(q))d)`，此外还有中心选择、Top-k 维护等开销。列表长度不同，同样的 `nprobe` 也可能对应不同的工作量。做实验时，需要同时看查询数和候选数。

2 已有代码

编译和运行方法见 README。实验机器的数据放在 `/var/dataset/`，程序已使用这个默认路径。配套代码可以运行以下两个串行版本：

- `search_ivf_serial`：串行 IVF 检索，作为主要对照。
- `search_exact_serial`：扫描整个数据库，用于检查搜索全部列表时的结果。

中心选择、标量距离、Top-k 容器、结果检查和重复计时也已提供。两个 OpenMP 函数和用于对比的 `search_std_thread` 都留在 `ivf_student.cpp` 中，需要自行补全；SIMD、任务划分、局部 Top-k 合并和实验中需要的统计也由学生完成。

索引的存储方式与 CSR 有些相似：

```
centroids[L * d]     列表中心
list_ptr[L + 1]      每个列表的起止位置
ids[N]               原始向量编号
vectors[N * d]       按列表存放的向量
```

第 l 个列表位于 `[list_ptr[l], list_ptr[l+1])`。检索时索引只读，不需要重新训练。文中的 Q 表示一次实验处理的总查询数，`batch` 表示一次函数调用最多处理多少个查询，最后一批可以少于 batch。

可以复用已有的辅助函数，但要保留串行参考和完整结果检查。最多增加到 5 个 `search_omp_student_*` 函数，额外实现多少个版本不直接加分。不要调用 Faiss 等库的多线程检索接口代替自己的 OpenMP 实现。

3 要完成的实现

3.1 按查询并行

完成 `search_omp_student_1`，把不同查询分配给不同线程。每个查询独立完成中心选择、候选扫描和 Top-k，使用自己的结果空间。

可以尝试 `static`、`dynamic` 等调度方式。测试时留意：batch 小于线程数，尤其是 `batch=1` 时，有多少线程能参与计算；不同查询的候选数差别较大时，平均分配查询数的效果如何。仅把循环调度方式换一下，不算完成第二种策略。

3.2 在查询内部划分任务

完成 `search_omp_student_2`，让多个任务一起处理同一个查询。基本过程是：

选择列表 → 划分候选块 → 每块计算局部 `Top-k` → 等待完成 → 合并结果

实现时需满足以下要求：

1. 每次 batch 调用只建立一个 OpenMP 线程团队，通过 `task` 或 `taskloop` 分配工作，不同查询的任务可以在同一个团队中执行。
2. 长列表也要能拆分。`grain` 表示每块最多包含的候选数，每个候选恰好计算一次。
3. 每块单独维护 Top-k。在合并结果、读取其他任务的输出或释放缓冲区之前，要保证相关任务已经结束。
4. 支持 `batch=1`。小任务可以直接计算，但如果设置了这样的阈值，需要用实验说明选择依据。

最终版本不能对每个候选都用一次 `critical` 更新共享 Top-k，也不能为每个查询再建立一套完整的嵌套线程团队。前一种写法可以保留为对照，观察同步开销。

报告中要说明每块只保留 k 个候选为什么足够，以及任务参数和局部结果放在哪里、由谁读写、何时释放。还要分析块数增加后，临时空间、调度和合并分别会付出什么代价。

3.3 SIMD 距离计算

使用 OpenMP `simd` 编写自己的候选距离函数，并接入上面两种实现。归约变量、数据类型和循环怎么组织，需要自行决定。保留原来的标量计算，添加一个开关，便于在相同条件下比较；开关的取值也要写入 CSV。

距离累加和 Top-k 更新的依赖关系不同，不能把 Top-k 更新直接放进距离归约循环。完成后，用编译器的向量化报告或反汇编检查是否生成了 SIMD 指令。去掉 `omp simd` 后，编译器仍可能自动向量化，因此报告中要根据实际编译结果描述比较对象。

SIMD 不一定会让整个检索过程更快。如果没有加速，结合距离计算所占时间、Top-k 开销等分析原因即可。

3.4 std::thread 对照版本

完成 `search_std_thread`，按连续区间划分查询，每次调用时创建线程，返回前完成计算并回收线程。线程创建和回收的时间也要计入。使用与串行版本相同的中心选择和距离函数，通过完整结果检查后，用 `--only std` 运行对比实验。

4 结果检查

比较并行实现时，数据库、索引、查询、k、nprobe 和中心选择规则都要相同。不能通过少扫描候选、提前结束搜索或修改参考结果来缩短时间。

每个查询的全部结果都要检查:

- 按 `(distance, id)` 升序排列，编号与串行参考一致。
- 距离比较采用 `atol=rtol=1e-10`。这个容差只用于检查结果，不要用在排序比较器中。
- 候选不足 k 个时，剩余位置填 `id=-1, distance=+inf`。

有数据竞争、未初始化读、越界，或只在部分线程数下正确的实现，不算通过。

SIFT 数据库和查询的坐标是整数，参考代码使用 double 计算差值、乘积和累加，可以精确表示本实验的距离。中心坐标是浮点数，各实现应共用已有的标量中心选择函数，以保证选中相同的列表。

请覆盖单线程和多线程、单查询和批量查询，也要检查空列表、候选不足 k、等距、`k=1`、`nprobe=1` 和 `nprobe=nlist`。可以用 `tests/ivf_tests.cpp` 中的小型数据补充自己的测试。现有 `make test` 只检查公共代码，不能代替对学生实现的验证。

当 `nprobe=nlist` 时，程序用独立的全库扫描检查结果；其他情况下，与串行 IVF 比较。另外，用 ground truth 的前 k 个编号计算 Recall:

$$\text{Recall@k} = \frac{1}{Qk} \sum_{q} |R_{\text{IVF}}(q) \cap R_{\text{GT}}(q)|.$$

这里两个集合分别表示 IVF 返回的编号和 ground truth 中的编号。固定 nprobe 时，正确的并行实现应得到相同结果和 Recall。调小 nprobe 虽然可能更快，但搜索范围也变了，这部分收益要与并行加速分开讨论。

公开 ground truth 在第 k 名出现等距时，编号顺序可能与本实验不同，因而全库检索的 Recall 也可能略低于 1。这不影响与串行参考的逐项检查，也不能用 Recall 接近 1 代替正确性检查。

5 性能实验

统一使用 SIFT1M: N=1,000,000, d=128, L=4096。默认 Q=128、k=10、nprobe=8, 加载时约需 1 GiB 内存。不要只截断数据库，却继续使用完整数据库的索引和 ground truth。

请完成下面五组实验，每组改变一个主要变量，不需要遍历所有参数组合。P 表示实验实际分配到的物理核心数。

实验	改变什么	测量和比较什么
A 线程数	p=1,2,4,...，最后补上 P	分别使用 batch=1 和 batch=4P，比较两种策略的时间、加速比和效率
B 查询工作量分布	同一查询池的原始顺序、按 C(q) 排序后的顺序	固定 p=P、batch、距离函数和 grain；同步重排 ground truth，比较工作量分布和时间
C 任务大小	grain=64,256,1024,4096	使用 omp2，其他参数不变，记录任务数、临时空间和时间，分析合并成本的变化

实验	改变什么	测量和比较什么
D 搜索范围	至少三个 nprobe，例如 1、8、32	其他参数不变，记录候选数、Recall 和 QPS
E SIMD	同一策略开启、关闭 SIMD	分别在 p=1 和 p=P 下测试，其他参数不变，附向量化报告或反汇编证据

A 中的大 batch 始终固定为 4P，不随 p 改变；若 Q<4P，就增加查询数。线程扩展主表统一使用公共标量距离函数。若另外报告 SIMD 版本相对标量基线的加速，要注明其中同时包含多线程和向量化的收益。

B 中至少给出列表长度、C(q) 的中位数和最大值。可以记录线程 t 完成的候选距离计算数 W_t，用 `max(W_t)/mean(W_t)` 观察工作是否分配均匀。各线程计数之和应等于总候选数。收集这类统计时另跑一遍，避免统计代码影响正式计时。排序后变快或变慢，都需要结合调度、数据局部性和任务开销解释。

还需选择一组大 batch 配置，比较 `std::thread` 和查询级 OpenMP 的实际运行时间。两者使用相同数据、距离函数和计时范围。报告中讨论每次创建、回收线程对短任务的影响，OpenMP 在循环调度上的便利，以及需要自己管理线程生命周期、任务队列或线程通信时，`std::thread` 的用途。

所有实验结果都要作简要分析。至少选一个重要结论，用消融实验或分阶段计时支持。若统计了各阶段时间，请说明是墙钟时间还是各线程工作时间之和；并发任务的耗时相加，通常不等于整次检索的耗时。

6 计时与报告指标

每个配置先预热至少一次，再正式重复 10 次，报告中位数、最小值和最大值。配套代码使用 `std::chrono::steady_clock`。

一次检索的计时应包含中心选择、候选划分、距离计算、Top-k、合并、工作区重置、线程管理和同步。数据加载、参考结果生成、检查和打印放在计时之外。不要把查询相关的计算提前移出去，也不要在重复运行时复用上次的答案；每次输出都要检查。

设处理全部 Q 个查询的时间为 T，单位为秒。T 是各次同步 batch 调用时间之和。使用以下指标：

$$\mathrm{QPS} = Q/T, \quad S(p) = T_{\mathrm{IVF,serial}}/T_{\mathrm{IVF,parallel}}(p), \quad E(p) = S(p)/p.$$

S(p) 的分子始终使用串行 IVF 时间，不使用全库扫描时间。线程数实验中保持距离函数相同；SIMD 开关的时间比单独报告。

`batch=1` 时，报告查询延迟的 p50、p95；`batch>1` 时，报告 QPS 和 batch 完成时间。batch 时间除以 batch 大小，得到的是每个查询分摊的开销，不能当作单查询响应时间。程序目前只输出每轮总时间、QPS、Recall 和检查结果，延迟分位数、候选数、实际线程数等统计需要自行添加。

各策略使用相同的编译选项。默认是 `-O3 -std=c++17 -fopenmp -pthread`，主实验不用 `-ffast-math`；如果使用 `-march=native`，请在报告中写明。还需记录编译器版本、CPU 型号、物理核数、SMT、内存、操作系统、OpenMP 运行时、可用 CPU 集合和线程绑定环境变量。

7 使用 Linux 性能工具

分别测量串行 IVF、一个 OpenMP 实现和 `std::thread`，至少选两个与分析有关的计数器解释结果。`lscpu`、`uname -a` 和 `perf list` 可以用来查看机器环境及可用事件。

程序的 `--only` 只选择检索函数，加载数据和计算串行参考仍会执行。因此，直接对整个进程运行 `perf stat`，得到的计数并不全是目标检索产生的。需要自己安排采集范围，例如使用 `perf stat --delay=-1 --control=...`：加载、验证和预热完成后开启计数，重复检索结束后关闭，并等待控制命令确认。检查计数是否覆盖了工作线程。配套代码尚未提供这部分脚本，也没有 `perf` 模式。

可以关注 task-clock、cycles、instructions、cache-misses、branch-misses、context-switches、cpu-migrations，并换算成每个查询或每个候选的计数。解释它们与任务调度、合并、SIMD 实验结果的关系。仅凭 cache-misses，不能判断内存带宽是否已经用满。

比较时，各版本使用相同的可用 CPU 集合。OpenMP 的绑定环境变量不控制 `std::thread`，要在报告中说明各版本实际如何绑定。事件不支持或发生 multiplex 时也应记录。若环境完全不允许使用 perf，保留报错，改用 `/usr/bin/time -v` 等工具，并说明测到了哪些信息、是否包含整个进程；不要填写没有测得的硬件数据。

8 提交材料与评分

提交一个压缩包，包含代码、复现命令、原始 CSV、性能工具输出、实验报告和心路历程。SIFT 数据文件较大，不用放进压缩包，但要注明使用了哪些数据和索引文件。

实验报告正文 2–4 页，可用 Markdown 或 LaTeX 导出 PDF。正文应包含：

1. 两种并行方法、局部 Top-k 合并和 SIMD 的实现思路。
2. 实验环境、参数、比较条件和正确性结果。
3. 时间、加速比、效率，以及 OpenMP 与 `std::thread` 的比较。
4. 查询工作量、任务大小、SIMD 的实验分析，以及 Recall 与 QPS 的关系。
5. 性能工具的数据和解释。
6. 最后的结论、遇到的 bug 或负优化，以及方法的局限和扩展性。
7. AI 使用声明和参考链接。

正确性截图和完整数据表可以放附录，但主要结论及其依据要在正文中说明。

另外提交 300–1500 字的心路历程，必须由学生本人完成，**不允许 AI 代写或润色**。写清楚方案是怎样选出来的，遇到了什么问题，做了哪些实验来判断原因，以及得到过哪些帮助。问过 AI 的技术问题要如实说明并附对话截图；查过的资料附链接，同学的帮助注明姓名。代码和实验报告可以使用 AI 辅助，但其中的数据和结论须由学生核实。

项目	分数	评分内容
基础要求：正确性	25	所有策略通过完整检查，无数据竞争或越界
基础要求：并行实现	25	完成两种 OpenMP、SIMD 和 `std::thread` 对照版本
基础要求：复现	10	材料齐全，命令、环境和原始数据足以复现实验
方案决策与探索	20	实验安排合理，消融和分析能说明方案选择的原因
性能工具	10	清楚说明计数测了哪部分代码，解释与结果相符
实验结果	10	相对合理基线的改进，以及不同配置下的稳定性和扩展性

评分看同一台机器上的比较，不按绝对 QPS 排名，也不按版本或图表数量加分。不是每个配置都必须加速；把负优化的原因分析清楚，同样可以获得探索分。正文超过 4 页、心路历程超过 1500 字等情况会扣分。心路历程若由 AI 辅助撰写，方案决策与探索部分计 0 分。

参考资料

- Faiss 索引说明
- OpenMP taskloop
- OpenMP taskgroup
- OpenMP SIMD
- perf stat